Programação de Aplicações Web

Aula 5: Animações, Boas Práticas e Frameworks CSS

9. Transições CSS (Suavidade Básica)

O estado original era brusco: botões piscarem ao tocar o mouse. O `transition` anima e suaviza propriedades de estado (como um `:hover` ou adição de `.ativo`) e garante elegância entre fases.

```
.botao {
  background-color: #3b82f6;
  color: white;
  transition: background-color 0.3s ease-in-out;
  /* 1. Qual propriedade (ou todas 'all'), 2. Tempo de Duração (300ms),
  3. Curva Bézier de velocidade (Timing function) */
}

.botao:hover {
  background-color: #1d4ed8;
  /* Quando o mouse entra, ele levará 0.3s até chegar nesse tom ciano escuro
  em vez de ser uma resposta imediata a piscar nos seus olhos. */
}
```

9. Transformações (2D e 3D)

O `transform` manipula caixas visualmente como transladar, escalar ou rodar, de forma altamente performática (rodando na GPU gráfica, fora do re-cálculo central).

- **Translate** `translate()`, movimenta usando coordenadas nos eixos (X,Y) baseadas em porcentagem do elemento pai ou unidade fixa: `transform: translateX(50px)`.
- **Scale** aumenta ou diminui o elemento: `transform: scale(1.5)`. Encolher ou inflar é mais fluido e elegante que tentar forçar o `width` ou `height` para animar o tamanho.
- **Rotate** rotaciona elementos nos ângulos: `transform: rotate(45deg);`
- **Skew** desvia e enviesa o elemento (usado muito para criar fundos inclinados em designs de cabeçalhos).

9. Animações Keyframes Completas (@keyframes)

Com a propriedade `animation`, a gente vai além das interações de `:hover`, criando nossos próprios movimentos, *quadro a quadro*, mesmo sem a interação do usuário.

```
@keyframes pulsarVermelho {
  0% { transform: scale(1); background: darkred; }
  50% { transform: scale(1.1); background: #ff0000; }
  100% { transform: scale(1); background: darkred;}
}

.alerta-emergencia {
  /* Animação - Duração, Quantidade Repetição (infinita),
  Alternar/Retroceder sentido da cena ou resetar loop */
  animation: pulsarVermelho 2s infinite ease-in-out;
}
```

10. Variáveis CSS (Custom Properties)

Variáveis ou Propriedades Customizadas definem localmente o armazenamento de valores para reuso dinâmico, facilitando MUITO o processo de criação visual (Imagine que você decida mudar a cor *Tema* do seu cliente do verde para o vermelho em 300 componentes diferentes do nada).

```
/* Escopo principal */
:root {
  --cor-primaria: #0d6efd;
  --espaçamento-base: 1.5rem;
  --sombra-card: 0 4px 6px rgba(0,0,0,0.1);
}

.botao {
  /* Chamamos via "var(chave)" */
  background: var(--cor-primaria);
  padding: var(--espaçamento-base);
  box-shadow: var(--sombra-card);
}
```

Isso garante uma forma brutal de gerenciar **Sistemas de Cores Complexos** e criação de **"Modo Escuro (Dark mode) "** na internet super simplificado, recriando as definições!

10. Tópicos Avançados (Metodologias)

Como criar nomes de `.classes` CSS em um projeto gigantesco sem colisões que quebrem a folha de estilo inteira, evitando dores de cabeça e manutenções complexas? Usamos metodologias (convenções) lógicas de organização:

Padrão BEM (Block, Element, Modifier)

Uma nomenclatura visual muito lógica e pragmática dividindo os componentes na tela de front-end.

`<Bloco>` _ `<Elemento>` - `<Modificador>`

```
<!— Componente do Menu Nav (Bloco) —>
<nav class="menu">
  <!— Pedaço intrínseco de dentro dele (Elemento) ! —>
  <a href="#" class="menu__link">Home</a>
  <!— Variação/Estado de alteração do item de dentro (Modificador) —>
  <a href="#" class="menu__link menu__link--ativo">Blog</a>
</nav>
```

Frameworks CSS

Escrever as propriedades do zero para um botão (padding, cor, tamanho da letra, borda arredondada) milhares de vezes começa a esgotar o desenvolvedor. Para isso serve o **ecossistema de Frameworks CSS**, que são bibliotecas com regras visuais criadas previamente e prontas para uso.

Frameworks de base para "Componentes Prontos":

- **Bootstrap:** Um dos mais influentes e famosos do mundo, com foco no sistema de grid de 12 colunas e componentes pré-estilizados. Exemplo: Você simplesmente aplica `<button class="btn btn-primary">` e um botão azul padronizado aparece.
- **Bulma:** Utiliza muito grid/flexbox e classes fáceis
- **Materialize / Material-UI (React):** Define designs influências na política e ecossistema UI e UX de animações e layout imposto pelo proprio *Google* da vida moderna no Android.

Tailwind CSS: Uma Filosofia Diferente (Utility-first)

Esqueça o paradigma antigo (escrever HTML, buscar classe "X" num arquivo CSS e ler 1500 linhas preenchendo as configurações). Essa é a força do **Utility-First** com TailwindCSS. A semântica cede espaço para uma construção altamente utilitária!

O Tailwind dispõe de uma coleção extensa de `classes em miniaturas que fazem apenas uma coisa isoladamente`.

```
<!— Botão roxo em Tailwind = padding-left/right(px-4), cima/baixo(py-2)
arredondado(rounded-lg), cor(bg-purple-600) → hover: sombra e escurece ... —>
<button class="bg-purple-600 hover:bg-purple-700 text-white font-bold py-2 px-4 rounded-lg">
  Sou um Botão Diferente
</button>
```

- **Você nunca sai do arquivo HTML** para dar "estilo". As classes se misturam e definem diretamente o design na tag.
- Durante o *build*, o Tailwind remove todas as classes não usadas e seu CSS gerado não pesará quase nada, ficando perfeitamente otimizado.

Tailwind CSS: Prós e Contras

Apesar de gerar certa polêmica na comunidade por adicionar muitas classes direto no HTML, o Tailwind é atualmente uma das bibliotecas CSS mais utilizadas e amadas pelas suas enormes vantagens:

Prós

- **Criatividade sem limites rígidos de temas:** Você não fica preso em componentes genéricos com "cara de Bootstrap". É possível criar um layout completo sem abrir arquivos CSS externos.
- Componentização em bibliotecas JavaScript modernas (como React) criam fluxos visuais poderosos em segundos.

Contras

- **O HTML pode ficar poluído visualmente:** Blocos podem ter dezenas de classes no próprio atributo (ex: `class="flex flex-col space-y-2 mt-4 max-w-lg lg:w-[40%] text-sm etc ... "`).
- **Curva de aprendizado:** Você precisa se acostumar com a nomenclatura única das classes utilitárias.